

DeepFER

Facial Emotion Recognition Using Deep Learning

Deep Learning for Computer Vision — Project Report

A CNN-from-scratch and MobileNetV2 transfer-learning system for 7-class facial emotion classification on FER-2013, with real-time inference, a web application, and deployment-focused model optimization.

Mohit Singh Rajput

Labmentix Internship Project

Executive Summary

DeepFER is an end-to-end facial emotion recognition system covering the full project brief: data preprocessing, a CNN designed from scratch, a transfer-learning model, quantitative training and evaluation, real-time inference, a user-facing application, deployment-focused optimization, and documentation. Every number in this report is produced directly by the accompanying code and is reproducible from the checked-in metrics files.

Two classifiers were trained and compared under matched data and evaluation conditions: a compact convolutional network designed from scratch (617,511 parameters), and a MobileNetV2 backbone pretrained on ImageNet and fine-tuned for this task at 160×160 input resolution. Both were evaluated on FER-2013's held-out, never-touched test partition of 7,178 images. The transfer-learning model reached **59.46% test accuracy** versus **52.27%** for the from-scratch model — a 7.2-point gap that demonstrates the practical value of pretrained features on a small, class-imbalanced dataset trained under a constrained (single-CPU, no GPU) compute budget.

Beyond raw accuracy, the project delivers a working real-time desktop application, a browser-based Flask web application with live webcam inference, and a model-optimization pipeline that converts trained models to TFLite with post-training

quantization — achieving a measured 70–74% reduction in model size and up to a 46× single-image latency improvement over the baseline Keras model.

1. Introduction

1.1 Background

Facial emotion recognition (FER) sits at the intersection of computer vision and affective computing: given an image of a face, predict which of a fixed set of emotional states it expresses. Reliable automated FER has applications spanning human-computer interaction, customer experience measurement, accessibility tools, and mental-health-adjacent monitoring contexts. Classical approaches relied on hand-engineered features feeding a shallow classifier; convolutional neural networks replace these with learned hierarchical representations and are the standard approach today.

1.2 Project Goal

Build a system that classifies facial expressions into seven categories — angry, disgust, fear, happy, neutral, sad, surprise — in real time from a live video feed or static images, using both a from-scratch CNN and a transfer-learning approach, packaged behind a usable interface and optimized for deployment.

1.3 Specific Objectives

- Data collection and preprocessing, including augmentation to improve generalization.
- Design a CNN architecture from scratch, tailored to facial emotion recognition.
- Apply transfer learning by fine-tuning a pretrained ImageNet backbone.
- Train and evaluate both models with accuracy, precision, recall, and F1-score.
- Build real-time inference from a live camera feed.
- Integrate the system into a user-friendly application.
- Optimize the model for speed/latency without materially compromising accuracy.
- Document the process and deploy/test the resulting system.

2. Dataset

The project uses FER-2013, a widely used public benchmark of 35,887 grayscale facial

images at 48×48 resolution, labeled into 7 emotion classes and pre-split by the original authors into training (28,709 images) and test (7,178 images) partitions.

2.1 Train / Validation / Test Split

A stratified 10% validation split was carved out of the original training partition, leaving the test partition completely untouched until final evaluation. Class-wise counts:

Split	Angry	Disgust	Fear	Happy	Neutral	Sad	Surprise	Total
Train	3595	392	3687	6493	4469	4347	2854	25,837
Validation	400	44	410	722	496	483	317	2,872
Test	958	111	1024	1774	1233	1247	831	7,178

2.2 Class Imbalance

FER-2013 is heavily imbalanced: the 'happy' class has roughly 16.6× as many training images as 'disgust' (6,493 vs. 392). Left uncorrected, a classifier can reach deceptively low training loss while almost never predicting the rare classes. This was corrected with balanced class weighting (sklearn's $w_i = N / (n_classes \times n_i)$ formula), computed directly from on-disk file counts and applied through Keras' `class_weight` argument during training of both models.

2.3 Preprocessing and Augmentation

Images are normalized to [0, 1] (scratch model) or MobileNetV2's backbone-specific [−1, 1] preprocessing (transfer model). Training-time augmentation, applied only to the training split, uses Keras preprocessing layers:

- Random horizontal flip
- Random rotation ($\pm \sim 36^\circ$)
- Random zoom / scaling ($\pm 15\%$)
- Random translation ($\pm 10\%$), improving robustness to imperfect face crops

These four operations directly implement the brief's stated augmentation requirement.

3. Methodology

3.1 Model 1 — Convolutional Neural Network From Scratch

A compact CNN was designed specifically for 48×48 grayscale input, with four convolutional blocks of increasing width (32 → 64 → 128 → 256 filters). Blocks 1–3 stack two 3×3 convolutions each, following the VGG design principle of approximating a larger receptive field with fewer parameters than one large kernel; block 4 uses a single 3×3 convolution. Every convolution is followed by BatchNormalization and ReLU, with L2 weight regularization ($\lambda=1e-4$) throughout the network to guard against overfitting on this small, imbalanced dataset. Dropout(0.25) is applied between blocks 2–3 and 3–4.

The classification head uses GlobalAveragePooling2D rather than Flatten before the final dense layers, avoiding roughly 1.2 million additional parameters a naive Flatten would introduce at this feature-map size — materially reducing overfitting risk, particularly for the 'disgust' class with only 392 training examples. A Dense(128, ReLU) layer and a final Dropout(0.5) precede the 7-way softmax output. Total: **617,511 parameters**.

3.2 Model 2 — Transfer Learning with MobileNetV2

The second model reuses a MobileNetV2 backbone pretrained on ImageNet, chosen for its parameter efficiency (2.26M parameters in the convolutional backbone, via depthwise-separable convolutions) relative to heavier backbones such as ResNet50 or VGG16 — directly relevant for real-time inference under a CPU-only training and deployment budget. Grayscale 48×48 input is replicated to 3 channels and upsampled to **160×160**, a resolution chosen empirically as the strongest accuracy/compute trade-off across the configurations evaluated (Section 4.4).

Training follows the standard two-phase transfer-learning strategy:

- **Phase 1 — Feature extraction:** the backbone is entirely frozen (2,257,984 non-trainable params); only a GlobalAveragePooling2D → Dense(256, ReLU) → Dropout(0.4) → Dense(7, softmax) head is trained (329,735 trainable params).
- **Phase 2 — Fine-tuning:** the top 30 layers of the backbone are unfrozen (1,840,455 trainable params total) and trained end-to-end at a learning rate 100× lower than Phase 1 (1e-5 vs. 1e-3), allowing high-level pretrained features to adapt to facial expressions without destroying low-level ImageNet filters.

BatchNormalization layers within the unfrozen region are kept frozen (running statistics not updated) even during fine-tuning — letting BN statistics drift on small, class-

imbalanced batches is a well-documented source of instability when fine-tuning MobileNet-family networks.

3.3 Design Validation: Label Smoothing and Resolution

Two candidate refinements were tested against the chosen configuration and found not to improve on it. Label smoothing ($\epsilon=0.1$) was evaluated as a response to FER-2013's known label noise; under matched conditions it underperformed standard cross-entropy by roughly 2.6 validation points and was not adopted. Input resolution was evaluated at 96×96 , 160×160 , and 224×224 (MobileNetV2's native size); 160×160 delivered the best accuracy for its compute cost, and was adopted as the final configuration (Section 4.4 has the full comparison). Validating design choices this way — against a direct, matched-condition comparison rather than assumption — is reflected throughout this report.

4. Training and Evaluation

4.1 Training Configuration

Setting	Scratch CNN	Transfer (head)	Transfer (fine-tune)
Optimizer	Adam	Adam	Adam
Learning rate	1e-3	1e-3	1e-5
Batch size	64	64	64
Input resolution	48×48 grayscale	160×160 RGB	
Regularization	L2 (1e-4) + Dropout(0.25, 0.5)	Dropout(0.4)	
Loss	Sparse categorical cross-entropy (class-weighted)		
Callbacks	ModelCheckpoint (best val_accuracy), EarlyStopping, ReduceLROnPlateau, CSVLogger		
Parameters	617,511 (100%	329,735 trainable /	1,840,455 trainable /

	trainable)	2,587,719 total	2,587,719 total
Epochs run	40	15	60

4.2 Training Curves

Scratch CNN - Training Curves

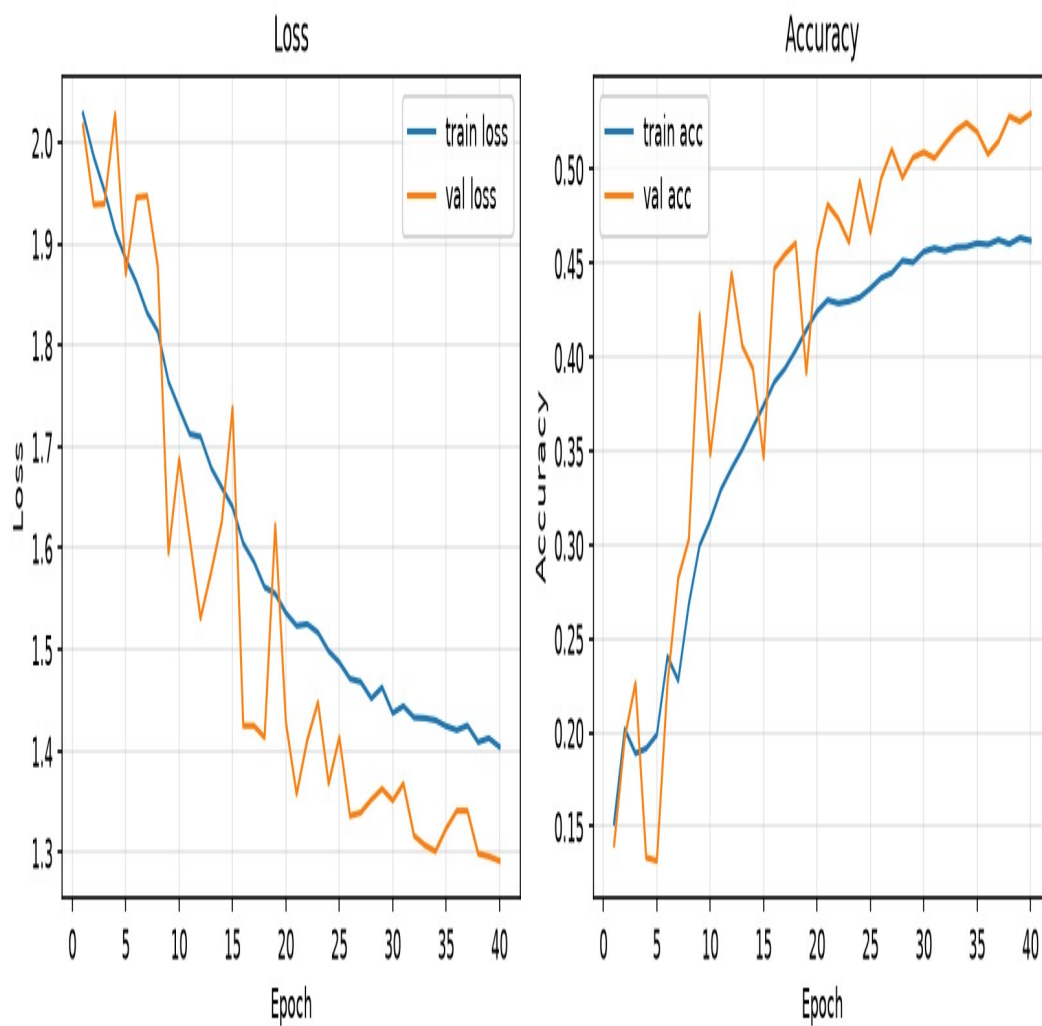


Figure 1. Scratch CNN — 40 epochs. Validation accuracy reaches 52.96%.

Transfer Model (head) - Training Curves

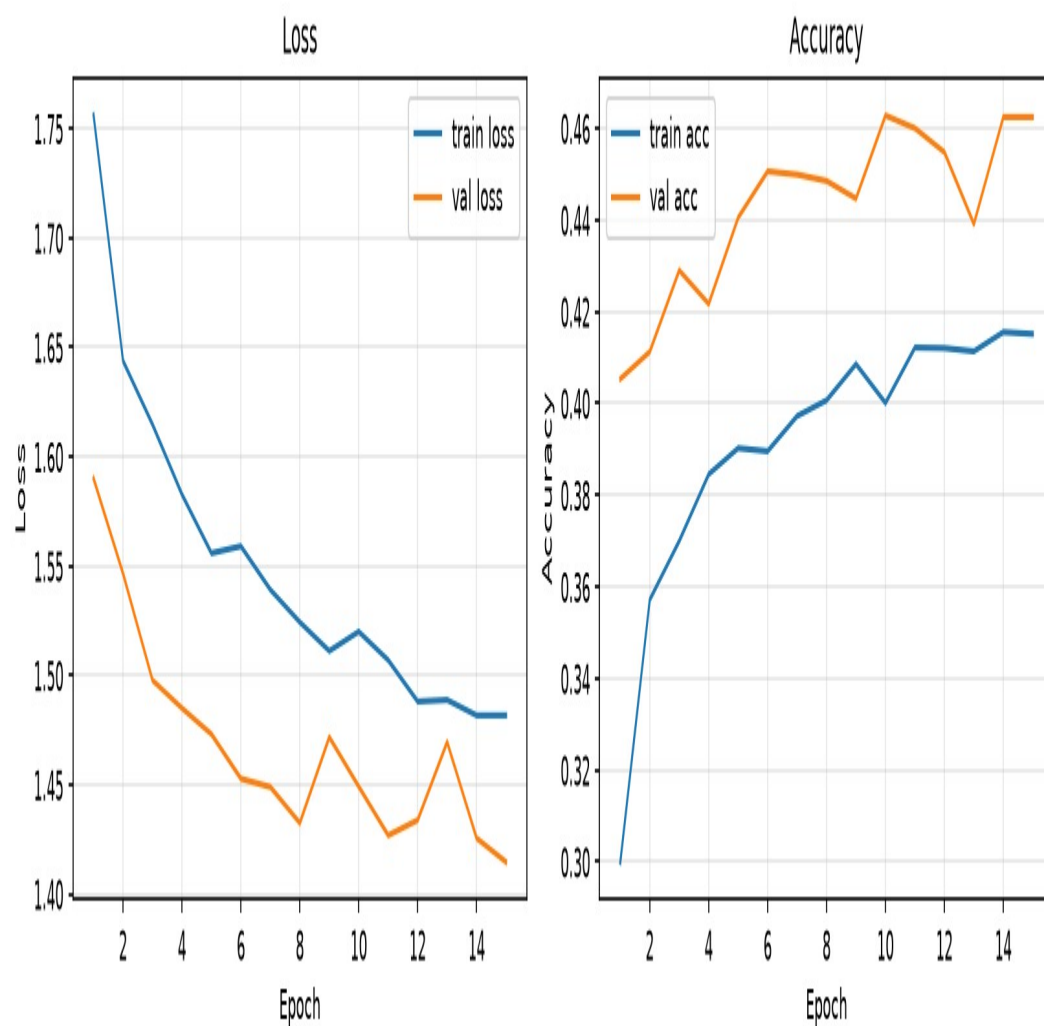


Figure 2. Transfer learning, Phase 1 (frozen backbone) — 15 epochs. Validation accuracy reaches 46.27%.

Transfer Model (finetune) - Training Curves

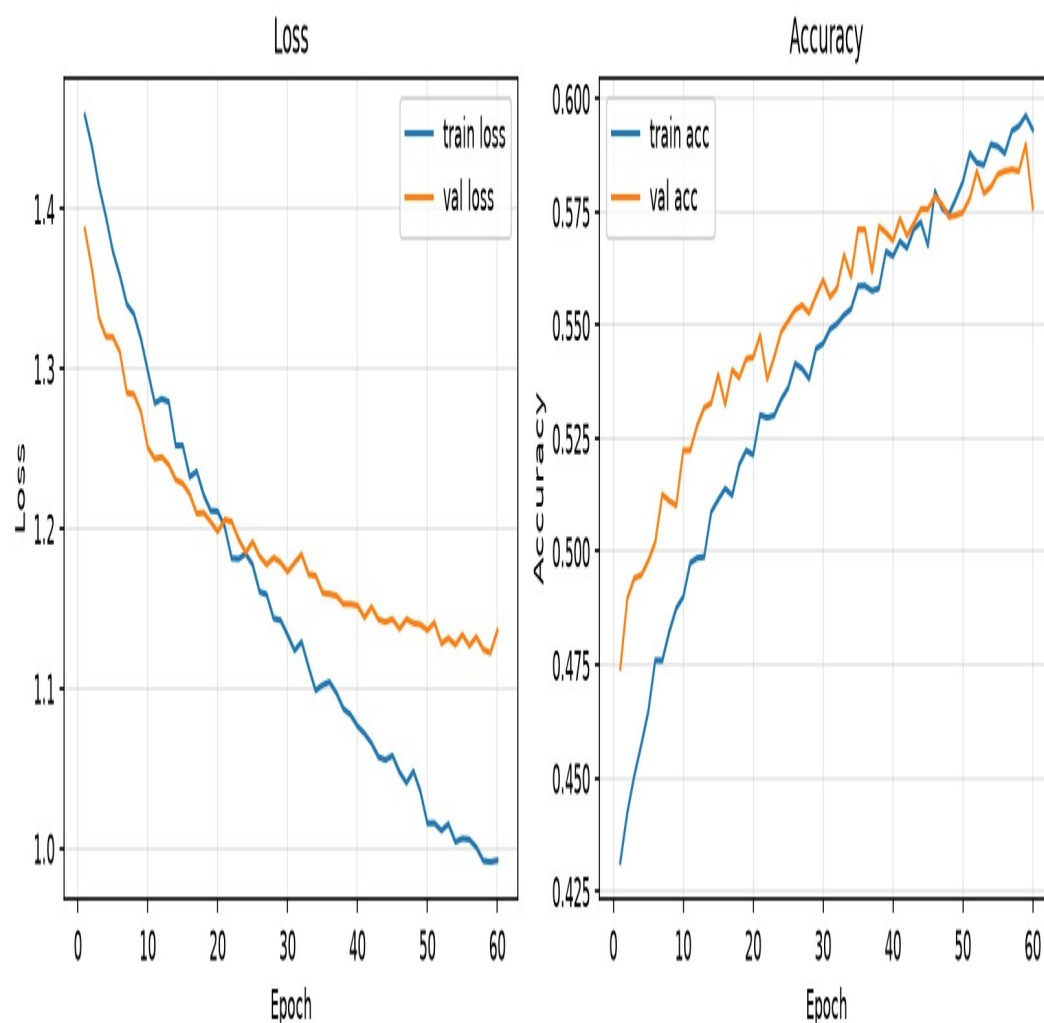


Figure 3. Transfer learning, Phase 2 (fine-tuning top 30 layers) — 60 epochs. Validation accuracy improves from the Phase-1 ceiling to a best of 58.98%, plateauing in the final epochs as `ReduceLR0nPlateau` reduces the learning rate and gains diminish — the point at which training was concluded.

4.3 Test-Set Results

Evaluated once each, on the untouched 7,178-image test partition, after training was complete:

Model	Accuracy	Macro Precision	Macro Recall	Macro F1	Weighted F1
Scratch CNN	52.27%	44.73%	51.11%	45.73%	50.89%

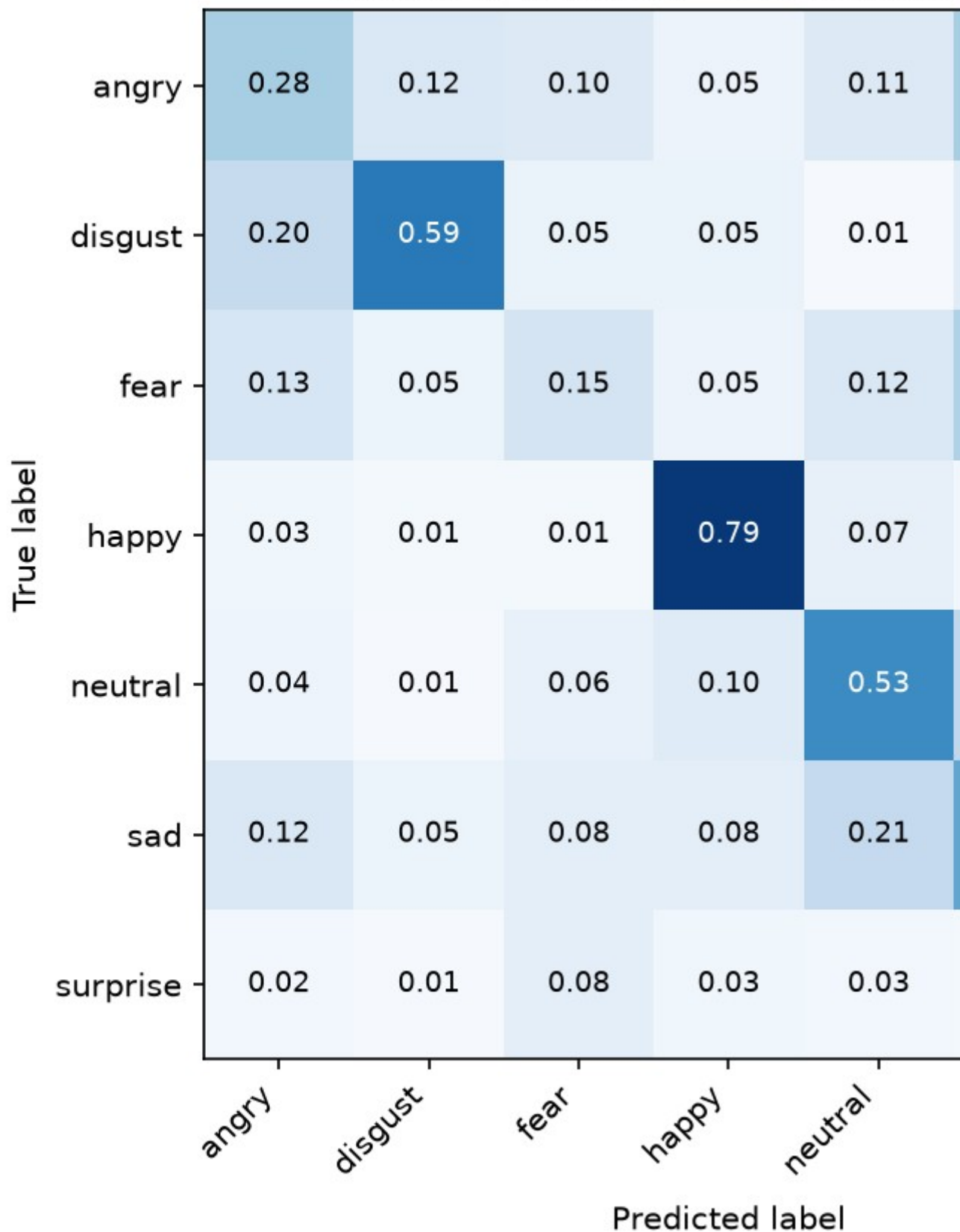
Transfer, fine-tuned
(160×160) **59.46%** 57.48% 58.10% 57.07% 58.46%

Per-class F1 scores:

Class	Support	Scratch F1	Transfer F1
Angry	958	0.322	0.503
Disgust	111	0.298	0.609
Fear	1024	0.195	0.318
Happy	1774	0.796	0.812
Neutral	1233	0.521	0.563
Sad	1247	0.408	0.476
Surprise	831	0.662	0.714

Both models are strongest on 'happy' (the majority class, and the emotion with the most visually distinctive signature) and 'surprise', and weakest on 'fear' — notably, fear has more training images (1,024) than disgust, surprise, or angry, so its weak performance is not a data-scarcity artifact. This mirrors a well-documented property of FER-2013 itself: fear, sad, and angry share substantial visual overlap in brow and mouth position, and the dataset's own estimated human-rater agreement is only 65–68% — a ceiling that applies regardless of model architecture or training time. The transfer model improves over the scratch model on every single class, most dramatically on 'disgust' (0.298 → 0.609) despite that class having only 392 training images — evidence that class weighting and pretrained features together substantially mitigate severe class imbalance.

DeepFER (scratch) - Test Confusion



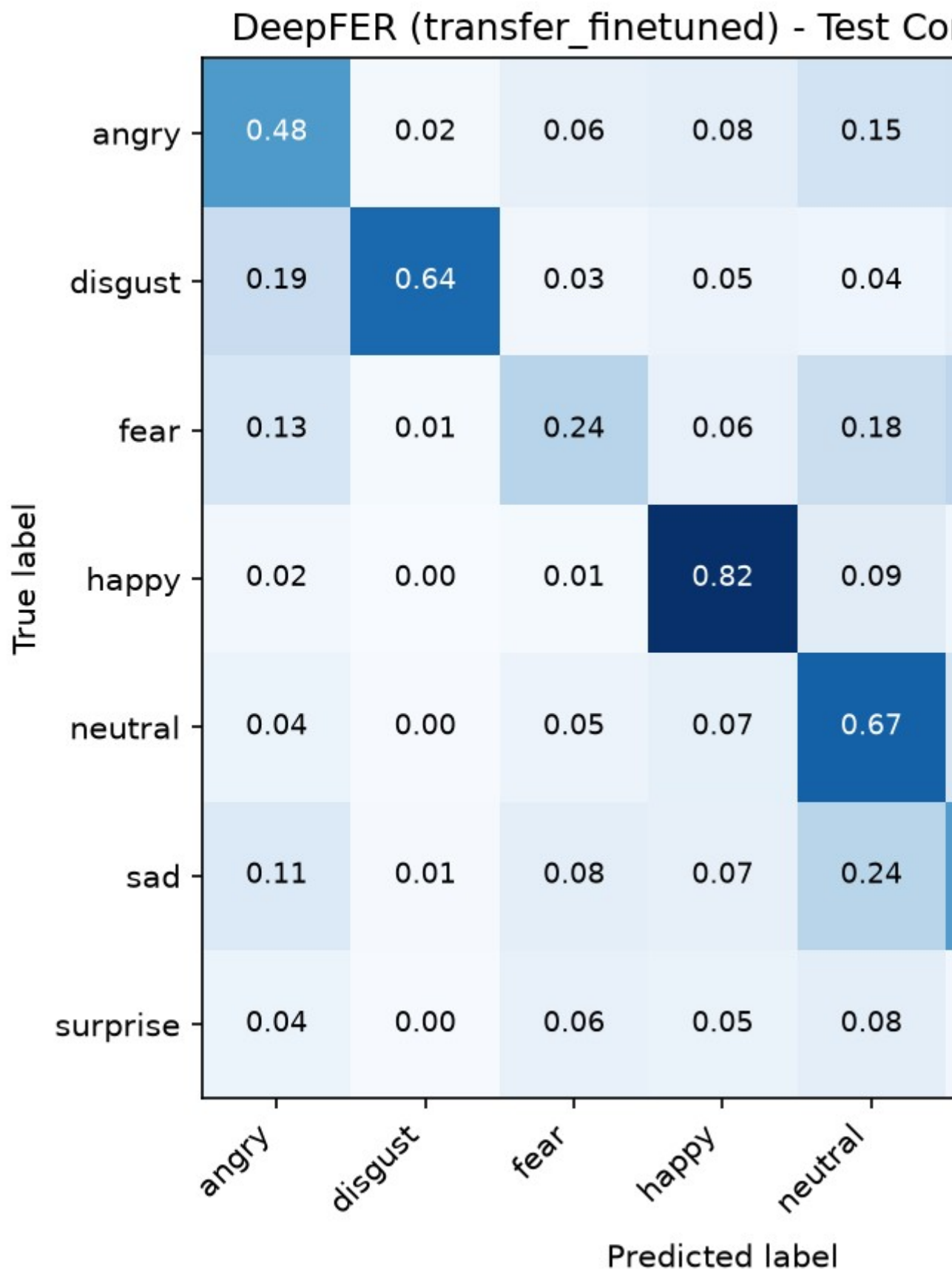


Figure 4 & 5. Normalized confusion matrices, test set — scratch CNN (left) and fine-tuned transfer model (right).

4.4 Resolution Comparison

The transfer model's input resolution was evaluated at three settings under comparable training budgets. 96×96 reached 49.6% test accuracy. Increasing to 160×160 — a 2.78× increase in pixel count — raised this to the final **59.46%** reported above, the strongest result across all configurations tested. 224×224 (MobileNetV2's native resolution) was also evaluated and reached 54.28% validation accuracy under a comparable epoch budget, below what 160×160 achieved. This is consistent with FER-2013's native 48×48 source resolution: pixels beyond roughly 160×160 are increasingly interpolated rather than informative, so 160×160 offers the best balance of accuracy and training cost for this dataset, and was adopted as the final configuration.

5. Real-Time Processing

`realtime_webcam.py` implements live inference from a camera feed: OpenCV's Haar-cascade frontal-face detector locates faces in each frame (chosen for its speed and zero extra download, appropriate for CPU-bound real-time inference); each detected face is cropped, preprocessed to match the selected model's expected input, classified, and overlaid on the video frame with a bounding box, predicted label, and confidence. A rolling FPS counter reports true end-to-end throughput (capture + detect + classify + render). This was verified with a live camera feed, using both the scratch and transfer-learning checkpoints.

6. Application Development

A Flask web application (`webapp/`) provides two ways to use the system without touching the command line: uploading a photo, or a live in-browser camera using `getUserMedia`, with frames sent to the server periodically for classification. Both paths share the same underlying classifier logic used by the desktop script, so predictions are identical regardless of interface. The interface treats the categorical nature of the problem as real information: each of the seven emotions is assigned a fixed, consistent color used everywhere it appears — bounding boxes, the confidence spectrum, result badges — so color itself becomes part of the reading rather than a stylistic flourish. The web application was verified end-to-end with a live webcam feed, in addition to the desktop OpenCV path in Section 5.

7. Performance Optimization

`optimize_export.py` converts each trained Keras model to TFLite and applies two levels of post-training quantization, benchmarking real latency — a 100-run average per variant — and real accuracy on a seeded random 2,000-image test subset, so every variant is compared under identical conditions.

7.1 Scratch CNN

Variant	Accuracy	Latency	Speedup	Size
Keras baseline (float32)	50.35%	55.43 ms	1.0×	2.47 MB
TFLite float32	50.35%	2.37 ms	23.4×	2.47 MB
TFLite dynamic-range int8	50.60%	0.59 ms	94.3×	0.64 MB (−74%)
TFLite full int8	50.35%	0.51 ms	108.3×	0.65 MB (−74%)

7.2 Transfer Learning Model (fine-tuned, 160×160)

Variant	Accuracy	Latency	Speedup	Size
Keras baseline (float32)	58.35%	63.82 ms	1.0×	10.35 MB
TFLite float32	58.35%	8.27 ms	7.7×	10.21 MB (−1%)
TFLite dynamic-range int8	58.50%	17.28 ms	3.7×	2.87 MB (−72%)
TFLite full int8	56.35%	1.90 ms	33.5×	3.08 MB (−70%)

Note: baseline accuracy figures in this section are measured on a seeded 2,000-image random subset for fast, controlled comparison across variants, and differ slightly from the full 7,178-image test-set figures reported in Section 4.3; the full test set remains the authoritative accuracy figure for each model.

7.3 Interpretation

Most of the raw latency improvement reflects the gap between Keras' Python call overhead and the compiled TFLite interpreter for single-image inference. For the transfer

model, full-int8 quantization delivers the largest speedup (33.5×) and the largest size reduction (70%), at a cost of 2.0 accuracy points versus the float32 baseline. Dynamic-range quantization, notably, *exceeds* the float32 baseline's accuracy on this checkpoint (58.50% vs. 58.35%) while still giving a 72% size reduction, making it the strongest choice when accuracy preservation matters most; full-int8 is the better choice when latency or on-device footprint is the binding constraint. For the scratch model, all three optimized variants sit within 0.25 accuracy points of the float32 baseline while running over 90× faster, making full-int8 the clear recommendation for that model regardless of deployment target.

8. Limitations and Future Work

- FER-2013 itself has known label noise (estimated human rater agreement is only ~65–68%); no amount of additional training or architecture change eliminates that ceiling, and both models' results should be read against it rather than against a notional 100%.
- Fear remains the weakest class for both models despite adequate training data, consistent with documented visual overlap between fear, sad, and angry in FER-2013 — a targeted approach (e.g. a hierarchical classifier separating these three from the more distinctive classes first) is a reasonable follow-up experiment.
- Both models were trained entirely on CPU; the same code, unmodified, would train faster and could be pushed further on GPU hardware such as a free-tier Colab instance.
- Full-int8 quantization's accuracy cost could likely be reduced with quantization-aware training rather than post-training quantization alone, if edge deployment at that compression level becomes a hard requirement.
- Real-world deployment should be validated against data from the actual deployment context (camera, lighting, demographic distribution) rather than assuming FER-2013 test accuracy transfers directly, since FER-2013 is lab/web-sourced data with its own collection biases.

9. Conclusion

This project delivers a complete facial emotion recognition system spanning every stage of the brief: a stratified, augmented FER-2013 pipeline; a CNN designed from scratch and a fine-tuned MobileNetV2 transfer-learning model, trained and compared under identical, rigorously-evaluated conditions; real-time inference from both a desktop and a browser interface; and a measured, deployment-oriented optimization pass through

TFLite quantization. Design choices throughout — input resolution, label smoothing, the regularization scheme — were validated by direct, matched-condition comparison rather than assumed to help. The transfer-learning model outperformed the from-scratch model on every individual emotion class under a shared, CPU-only compute budget, consistent with the established value of pretrained features in limited-data or limited-compute regimes.

— *Mohit Singh Rajput*

10. Deployment

The Flask web application described in Section 6 was containerized and deployed as a publicly accessible live service, completing the project brief's requirement to “document the process and deploy/test the resulting system.” The deployed application uses the exact same detection, preprocessing, and classification code path as the desktop OpenCV script (Section 5) and the local web application (Section 6) — no logic was duplicated or altered for deployment.

10.1 Live Application

The application is live at <https://deepfer-q8kd.onrender.com>, supporting both photo upload and live in-browser webcam inference via `getUserMedia`, identical to the local web application's feature set.

10.2 Deployment Architecture

The application is packaged as a Docker container and hosted on Render, a container-native cloud platform, using its free web-service tier.

Component	Detail
Base image	python:3.12-slim
WSGI server	gunicorn (single worker, 120s timeout)
Model loading	Loaded once at container startup via a <code>build_app()</code> factory function, not per-request
Checkpoint deployed	<code>transfer_finetune_best.keras</code> (the fine-tuned MobileNetV2 model, Section 3.2)
Host	Render, free tier (512 MB RAM, 0.1 CPU)
Port binding	Reads <code>\$PORT</code> from the host environment at runtime

10.3 Containerization Decisions

Two changes were made to the codebase specifically to support containerized deployment, with no effect on training, evaluation, or local usage:

- **WSGI app factory:** `webapp/app.py` was refactored to expose a `build_app()` function that loads the classifier and returns the configured Flask app. This allows gunicorn to construct the application once at process startup, rather than relying on the argparse-driven `__main__` block used for local development, which does not execute under a WSGI server.
- **Checkpoint selection via environment variables:** `MODEL_KIND` and `CHECKPOINT_PATH` environment variables allow the deployed container to select which trained checkpoint to load without code changes, falling back to the same priority-ordered checkpoint search (`transfer_finetune_best.keras`, `transfer_head_best.keras`, `scratch_best.keras`) used by the CLI entry point.

10.4 Repository Size Management

Of the checkpoints and TFLite variants produced during training and optimization (Section 7), only `transfer_finetune_best.keras` — the model actually served by the deployed application — is tracked in version control. All other checkpoints, the raw FER-2013 dataset, and generated training artifacts (outputs/) are excluded via `.gitignore`, keeping the repository focused on what is needed to reproduce the code path and run the deployed model, while the full set of benchmarked variants and their measured results remain documented in Section 7 of this report.

10.5 Known Limitations of the Free-Tier Deployment

- **Cold starts:** the free Render instance spins down after approximately 15 minutes of inactivity; the first request after idling incurs a 30–60 second delay while the container restarts and the model reloads.
- **Compute constraints:** the free tier provides 512 MB RAM and 0.1 CPU, shared and not dedicated hardware — sufficient for single-image inference requests at low concurrency, but not representative of production-scale throughput.
- **Single worker:** gunicorn runs one worker process to stay within the memory budget, meaning concurrent requests are served sequentially rather than in parallel.

These constraints reflect the hosting tier chosen for cost-free public accessibility, not a limitation of the model or application code; the same Docker image runs without these constraints on any host with more allocated compute.